

Lab Assignment No.	04
Title	Write code to simulate requests coming from clients and distribute them among the servers using the load balancing algorithms.
Roll No.	
Class	BE AI & DS
Date Of Completion	
Subject	Computer Laboratory III[417534]
Assessment Marks	
Assessor's Sign	

Assignment No. 04

Title: Simulation of Client Requests and Load Distribution Using Load Balancing Algorithms

Problem Statement: To design and implement a simulation of client requests and distribute these requests among multiple servers using various load balancing algorithms such as Round Robin, Least Connections, and Random Selection in Jupyter Notebook.

Prerequisites:

- Basic knowledge of Python programming
- Understanding of networking concepts
- Familiarity with load balancing algorithms
- Basic knowledge of Jupyter Notebook environment

Software Requirements:

- Jupyter Notebook
- Python 3.x

Hardware Requirements:

- A computer with minimum 4 GB RAM
- Dual-core processor or higher
- Stable internet connection

Theory: Load balancing is a technique used to distribute network or application traffic across multiple servers. This ensures no single server bears too much demand, enhancing responsiveness and availability of applications.

Load balancing can be broadly categorized into two types:

1. **Static Load Balancing:** In this method, the distribution of traffic is predefined and does not change based on server conditions. It works well for systems with uniform load distribution.
2. **Dynamic Load Balancing:** This method adapts to changing conditions in real-time, distributing traffic based on current server load, response time, or other metrics.

Common Load Balancing Algorithms:

1. **Round Robin:** Requests are distributed cyclically to all servers. This is simple to implement and works well when all servers have similar capabilities.
2. **Least Connections:** Requests are sent to the server with the fewest active connections. This algorithm is efficient in environments where requests have varying processing times.
3. **Random Selection:** Requests are assigned to a randomly selected server. It is easy to implement but may lead to uneven load distribution if not managed properly.

Importance of Load Balancing:

- Enhances system reliability and availability
- Improves application performance
- Reduces server downtime
- Efficient utilization of resources

Source Code –

```
import random

import time

class Server:

    def __init__(self, id):

        self.id=id

        self.current_connections = 0

    def handle_request(self):

        """Simulate handling a request by a server."""

        self.current_connections += 1

print(f"Server {self.id} is handling the request. Total connections: {self.current_connections}")

    def release_request(self):

        """Release a request from the server."""
```

```
        If self.current_connections > 0:

            self.current_connections -= 1

        print(f"Server {self.id} has released a request. Total connections
{self.current_connections}")

class LoadBalancer:

    def __init__(self, servers):

        self.servers = servers

        self.server_count = len(servers)

        self.round_robin_index = 0

    def round_robin(self):

        """Round Robin load balancing algorithm."""

        server = self.servers[self.round_robin_index]

        self.round_robin_index = (self.round_robin_index + 1)% self.server_count

        return server

    def least_connections(self):

        """Least Connections load balancing algorithm."""

        Server = min(self.servers, key=lambda s: s.current_connections)

        return server

    def random_selection(self):

        """Random load balancing algorithm."""

        return random.choice(self.servers)

    def distribute_request(self, algorithm="round_robin"):

        """Distribute requests among servers based on the selected algorithm."""

        If algorithm == "round_robin":

            server = self.round_robin()
```

```
elif algorithm == "least_connections":
    server = self.least_connections()

elif algorithm == "random":
    server = self.random_selection()

else:
    raise ValueError("Unknown algorithm")

    server.handle_request()

# Simulate the scenario

def simulate_requests():

    servers = [Server(i) for i in range(1, 4)]    # Three servers

    load_balancer = LoadBalancer(servers)

    # Simulate 10 client requests

    for _ in range(10):

        algorithm = random.choice(["round_robin", "least_connections", "random"])
        # Randomly select an algorithm

        print(f"\nDistributing request using {algorithm} algorithm.")

        load_balancer.distribute_request(algorithm)

        time.sleep(0.5)    # Simulate time delay between requests

        # Simulate releasing requests

        For server in servers:

            server.release_request()

# Run the simulation

simulate_requests()
```

Conclusion - Through this practical, we have explored the fundamental concepts of load balancing and its significance in distributed systems. By simulating client requests and implementing various load balancing algorithms such as Round Robin, Least Connections, and Random Selection, we observed how different strategies impact the distribution of traffic among servers.